Citi Bike Archive: Local Processing to Databricks

Implementation workflow for VS Code Codex
CUNY 622 | October 1, 2026

Build a reproducible pipeline that inventories the historical Citi Bike archive, normalizes selected trip files into local Parquet, generates station-demand features, and publishes verified data to the class Databricks workspace. Use these features alongside class station snapshots to predict 30-minute empty/full risk.

How to use this PDF

Save this PDF in the project's docs folder. Open the project in VS Code and give Codex the handoff instruction below. Execute stages in order, checking each completion criterion. The Mac paths below are intended for your local VS Code session; this ChatGPT session has not inspected the archive.

Handoff instruction: Read docs/Citi_Bike_Local_to_Databricks_Codex_Workflow.pdf. Implement the workflow one stage at a time, starting with project setup and a read-only archive inventory. Inspect existing repository instructions first. Record decisions and progress in docs/implementation_status.md. Complete local work before requesting workspace details needed for cloud execution. Preserve source files and do not invent schemas, table names, or completeness claims.

Storage decisions

Layer	Location / purpose
Original archive	/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject - immutable CSV/ZIP sources.
Code project	Use the GitHub Classroom repository if available; otherwise a separate citibike-production subfolder.
Processed data	/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject_processed - Parquet, manifests, staging, and quality outputs.
Local query engine	DuckDB reads Parquet directly. A small local database may store views/metadata; do not duplicate the full archive into PostgreSQL.
Class workspace	Unity Catalog volume for uploaded files; managed Delta tables for shared data, features, predictions, and metrics.

Databricks itself runs in the cloud. Start with DuckDB and Parquet locally; local Spark is optional if profiling shows it is useful. Keep Databricks Connect in a separate environment from any local PySpark installation.

Bounded first milestone

Inventory the full archive; normalize one complete modern month and one historical sample month; then process a complete recent year. Expand to older years only after the pilot reconciles and the modeling question justifies it. No full-archive pandas load.

Stage 0 - Project setup and operating rules

Use the Classroom repository as the code root when it exists. Otherwise create the local code directory shown below. Keep generated data outside Git and outside the archive scanner's input set.

```
mkdir -p "/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject/citibike-production"
cd "/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject/citibike-production"
python3 -m venv .venv-local
source .venv-local/bin/activate
python -m pip install --upgrade pip
python -m pip install duckdb pyarrow pandas pyyaml pytest
```

Before using these commands, check installed Python and compatibility. Record Python and package versions in the project and pin the resolved dependencies. Modeling, dashboard, and MLflow dependencies can be added after data preparation works.

Project item	Implementation responsibility
config/local.yaml	Source root, processed root, pilot months, memory/thread limits, schema version.
src/citibike_pipeline/	Inventory, normalize, validate, aggregate, station history, cloud publishing.
tests/	Representative legacy/modern fixtures and failure/rerun checks.
docs/	Data dictionary, schema mappings, quality policies, progress, runbook.
databricks/	Workspace scripts, bundle configuration, dev/prod job definitions.
.gitignore	Exclude environments, data, caches, DuckDB files, logs, credentials, and local generated artifacts.

Configuration and resource policy

Inspect RAM, free disk space, architecture, installed tools, repository instructions, and source-folder accessibility. Default to conservative memory and concurrency; allow DuckDB disk spill under the processed directory. Estimate disk needs from the pilot rather than assuming compression ratios.

The scanner must explicitly exclude the code project, processed outputs, temporary folders, and hidden/system artifacts. Only supported source CSVs and CSV members of ZIPs are candidates. Preserve originals; stage output to a run-specific directory and publish only after validation.

Instruction to VS Code Codex

Create the project scaffold and configuration. Check the source path without changing it. Implement a CLI with subcommands inventory, normalize, validate, aggregate, and publish-plan. Use configuration files and documented arguments instead of hard-coded workspace values. Add --dry-run for mutations and resumable per-source processing. Report the setup and available resources.

Completion criterion

The environment imports successfully; the CLI help works; paths resolve; generated files are excluded from Git and source scanning. No archive conversion or cloud upload has occurred.

Stage 1 - Inventory the archive before conversion

Separate a fast discovery pass from a full profiling pass. Discovery lists source candidates, ZIP members, headers, sizes, and nominal months. Full profiling streams selected/all candidates for row counts, date coverage, parse errors, and checksums.

Manifest field	Required meaning
source_key / path / member	Stable identity: relative CSV path or ZIP path plus member. Record container and member separately.
size / modified time / SHA-256	File metadata and content identity; member content hash detects extracted copies.
schema fingerprint / family	Actual original header and ordered mapping; classify by header, not year alone.
nominal release / part	Filename-derived month/part; keep separate from actual trip timestamps.
rows / parse failures / date range	Count parsed records, malformed records, missing/invalid start dates; min/max actual dates.
status / duplicate group	Discovered, profiled, selected, excluded, failed; reason and overlap findings.
run / parser version	Reproducibility and resume state.

ZIP and duplicate handling

Do not blindly ingest ZIPs and extracted CSVs. Compare member-content hashes against standalone CSV hashes and ingest one representative of exact duplicates. Different hashes may still contain overlapping records; flag overlaps for later investigation. Avoid indiscriminate unzip of the full archive; stream or extract selected members to staging without overwriting originals.

Completeness policy

Group all parts of a nominal release, but assign actual trip months by timestamps. A filename or consecutive observed dates is not proof of completeness. Cross-check available source release listings or expected parts where possible. Mark completeness confirmed, incomplete, or unverified and record the basis.

Instruction to VS Code Codex

Implement inventory with discovery and profiling modes. Enumerate CSVs and CSV ZIP members, classify actual headers, compute streamed hashes/row counts/date coverage, and write a machine-readable manifest plus a concise report. Exclude project/generated paths. Identify exact duplicate copies and candidate overlaps without deleting anything. Recommend pilot months, with explicit completeness confidence.

Completion criterion

Manifest can be resumed; every candidate has a status; duplicate ingestion risks are visible; pilot choices are justified. Unknown schema families are reported, not silently treated as modern or legacy.

Stage 2 - Normalize selected months to Parquet

Read each schema family with explicit string-first mappings and validated types. Keep original provenance and retain quality flags. Do not force every source into one inferred CSV schema.

Canonical field	Mapping / policy
started_at_local / ended_at_local	Legacy starttime/stoptime; modern started_at/ended_at. Parse seconds/fractions; preserve source wall time.
start_station_id / end_station_id	Strings, including decimal-looking IDs such as 6322.01; do not equate IDs across eras automatically.
station names / coordinates	Map both schemas; numeric casts with invalid-value flags.
membership_group / original_membership	Subscriber -> member; Customer -> casual. Preserve original labels and unknown values.
ride_id / rideable_type	Modern values when supplied; null in historical sources.
duration_seconds	Derive from timestamps; retain source duration separately and compare where supplied.
source_key / source_row / schema_version	Stable record provenance, original row position, normalization version.
quality_flags / analysis_eligibility	Missing endpoints, timestamp errors, coordinates, durations; task-specific exclusion policy.
trip_year / trip_month	Derived from valid local start timestamp; invalid-date records in a separate quarantine dataset.

Time, quality, and deduplication

Validate source timezone conventions. Preserve local times; flag ambiguous/nonexistent daylight-saving timestamps rather than silently assigning UTC. Treat legacy \N and other sentinel values as missing. Do not remove plausible long rides without a documented task-specific rule.

Deduplicate exact file copies first. Check modern ride IDs for identical copies versus conflicting records. Historical field equality is not proof of duplication; flag candidate duplicates and avoid collapsing distinct legitimate rides. Document every removal and retained conflict.

Output and reruns

Write a consistent Arrow/Parquet schema under trips/trip_year=YYYY/trip_month=MM. Start with moderate file sizes, benchmark on the pilot, and avoid partitioning by station or day. Use deterministic source/run tracking and atomic publish markers or validated directory swaps. Reruns must not append duplicate records. Late files require rebuilding affected partitions from the authoritative selected sources.

Instruction to VS Code Codex

Implement schema adapters from discovered headers. Normalize only approved pilot sources to staging Parquet with provenance, quality flags, and quarantine. Make publishing idempotent and interruption recovery explicit. Add tests for legacy/modern mappings, sentinel nulls, string station IDs, fractional timestamps, DST handling, duplicate conflicts, and reruns.

Completion: selected-source rows reconcile to published plus quarantined rows and documented deduplication; rerun counts are unchanged; schema is consistent across pilot partitions.

Stage 3 - Query, validate, and build demand features

Local query example

DuckDB supports direct Parquet queries and Hive-style partition fields. Use parameter binding for paths in application code; the SQL below illustrates a fixed path. [1-2]

```
SELECT trip_year, trip_month, count(*) AS trips
FROM read_parquet(
 '/Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject_processed/trips/*/*/*.parquet',
 hive_partitioning = true
)
GROUP BY 1, 2 ORDER BY 1, 2;
```

Quality report before scaling

Compare per-source and per-month counts, actual date ranges, membership/bike-type distributions, missing IDs, coordinate ranges, duration summaries, ride-ID conflicts, and station counts. Match source counts with normalized/quarantined/deduplicated accounting. Profile elapsed time, peak memory where measurable, disk footprint, and spill usage.

Only after pilot validation, process the chosen complete recent year. Full-history processing is a later batch with resume and monitoring; it is not required to obtain the first model.

Derived table	Contents / cautions
station_hour_activity	Aggregate departures by origin/start hour and arrivals by destination/end hour. Keep them separate.
station_month_activity	Departures, arrivals, observed days, and data-completeness indicators.
station_location_observations	ID/name/location evidence and dates. Candidate identity mappings require validation.
historical_demand_profiles	Station/hour/weekday departure, arrival, net-flow patterns computed only from preceding history.
station_capacity_utilization	Join effective-dated capacity where supported. Never apply present dock counts to all historical years.

Availability-model integration

Use only finalized trip history available by each forecast origin; historical trip releases arrive later than the rides they describe. For backtesting, simulate release availability or explicitly label retrospective features. Class snapshots provide current counts and future labels; trips cannot reconstruct historical inventory or unmet demand.

Build zero-trip hours only over supported/inferred station-active periods and report uncertainty. Cross-era station IDs and moves need a versioned crosswalk; proximity alone is insufficient proof of identity.

Instruction to VS Code Codex

Implement local validation and aggregates for the pilot, then the approved recent year. Build as-of historical profiles with release-availability metadata, documented station-active assumptions, and tests preventing future leakage. Report throughput and remaining uncertainties before expanding the archive.

Completion: aggregate totals reconcile; feature cutoffs are tested; unknown station identity/capacity is visible rather than fabricated.

Stage 4 - Publish verified data to Databricks

Obtain the class workspace URL, assigned compute/runtime, permitted catalog/schema, and table names. Audit existing class snapshots and weather/trip sources. Upload only data missing or genuinely useful; avoid duplicating instructor-provided history.

Authentication and landing location

```
databricks auth login \
  --host "https://YOUR-WORKSPACE.cloud.databricks.com" \
  --profile CUNY622
databricks current-user me --profile CUNY622
```

Choose the granted schema; do not assume main.cuny622 exists. Create a Unity Catalog volume if privileges permit or use the assigned volume. Local CLI paths use dbfs:/Volumes/...; notebook access uses /Volumes/.... [3-4]

```
databricks fs cp \
  "/LOCAL/PILOT_PARQUET_DIRECTORY" \
  "dbfs:/Volumes/CATALOG/SCHEMA/VOLUME/pilot_v1" \
  --recursive --profile CUNY622
```

The paths above are placeholders. Generate an upload manifest and reconcile file hashes/counts after transfer. Begin with a versioned pilot prefix. Move to S3 only if the class configuration warrants it; do not provision a separate paid workspace or bucket by default.

Delta table creation in a workspace notebook

```
df = (spark.read
      .option("basePath", "/Volumes/CATALOG/SCHEMA/VOLUME/pilot_v1")
      .parquet("/Volumes/CATALOG/SCHEMA/VOLUME/pilot_v1"))
df.write.format("delta").mode("errorifexists").saveAsTable(
    "CATALOG.SCHEMA.citibike_trips_pilot_v1"
)
```

Use the agreed table identity and an explicit schema. For full publication, implement reviewed partition replacement or merge semantics, not repeated append or unrestricted overwrite. Validate counts and key aggregates before promoting tables.

Shared external-data requirement

Publish documented ACS transportation-access indicators and a station-to-tract crosswalk: GEOID/vintage, households without vehicles, estimates/margins of error, retrieval date, provenance, field definitions, and intended uses. Keep neighborhood context distinct from rider identity.

Instruction to VS Code Codex

Inspect class access and existing sources. Produce a publish plan with destination identifiers and expected counts; upload the pilot to the agreed volume, create a versioned Delta table, and reconcile it. Implement reruns without duplicates. Publish the external dataset with table/column documentation. Keep local and Databricks Connect environments separate.

Stage 5 - Train and operate the production MVP

Core model: two binary classifiers predicting no rentable bikes and no usable return docks at t + 30 minutes. Score operating stations every 15 minutes using class snapshots, available weather, and historical profiles. Financial estimates and ACS access context support interpretation.

Labels and training

Use observation/reporting times and collection times distinctly. Match target observations within a documented tolerance; do not match across long gaps. Exclude unavailable/stale labels. Resource shortages and disabled service are different outcomes. Compare persistence, logistic regression, and gradient-boosted trees using chronological splits and matured labels.

Evaluate precision-recall AUC, Brier score, calibration, and precision/recall among a fixed alert budget. Report sample counts, prevalence, coverage, busy/quiet station results, and neighborhood breakdowns. Test whether trip-history features improve snapshot-only predictions.

Production component	Expected behavior
Feature + prediction job	Every 15 minutes; freshness gate; deterministic key (station, origin, horizon, model version); write before outcomes.
Outcome evaluation	Daily; match matured outcomes and retain unmatched reasons. Store metrics by origin date and model version.
Training job	Weekly; train/evaluate candidates, apply promotion criteria, retain a working previous model.
Dashboard	Current/predicted risk, freshness, neighborhood access, observed reliability; no invented savings.
Monitoring	Missed jobs, stale feeds, prediction coverage, failures, scoring time, calibration, and alert accuracy.

Deployment and financial scope

Define jobs in Databricks Declarative Automation Bundles, with isolated dev/prod destinations. Validate and run dev before enabling production schedules. Compute and authentication must follow the class setup. Bundles support local source-controlled job deployment. [5]

```
databricks bundle validate -t dev --profile CUNY622
databricks bundle deploy -t dev --profile CUNY622
```

Compare station use and cost per departure under explicit capital/operating assumptions. Snapshot changes cannot identify all rebalancing, and forecasts cannot establish causal benefits of adding docks. Treat intervention benefits and savings as scenarios.

Instruction to VS Code Codex

Build the snapshot-based baseline end to end, then test historical features. Store predictions, labels, metrics, and model lineage. Create a dashboard and dev jobs, simulate stale inputs/retries, and document recovery. Enable production schedules only after the system and class permissions are verified.

Completion: live scheduled prediction -> persisted output -> later outcome -> reproducible metric; a demo works without manual data edits.

Stage gates, deadlines, and first Codex task

Gate / deadline	Evidence to retain
Local pilot	Manifest, mapping, quarantine report, reconciled counts, rerun test, resource profile.
October 4 proposal	At most three pages plus references; seven required questions and bounded MVP.
Recent-year features	Verified periods, as-of cutoffs, station identity limitations, aggregate reconciliation.
Cloud pilot	Uploaded manifest, Delta reconciliation, class data audit, external-table documentation.
November 2-7 MVP	Three slides and five-minute end-to-end demo; baseline performance and remaining risks.
November 16-December 14	Four weeks of stored forecasts, outcome matches, reliability logs, and production metrics.
December 18 documentation	Model card, external-data datasheet, runbook, and ~2,000-word technical report.

Suggested first message to Codex

Use this PDF as the implementation specification. Start with Stages 0 and 1 only. The source archive is /Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject. Prefer the existing Classroom repository if present. Put generated data in /Volumes/BigRaid/RandyFiles/Cuny/622/FinalProject_processed and exclude all generated/code folders from scanning. Inspect resources and discover CSV/ZIP schemas. Build a resumable inventory and report duplicates, coverage, and pilot-month choices. Do not convert the entire archive, delete sources, upload data, or guess missing workspace settings. Keep progress in docs/implementation_status.md and provide exact commands to reproduce completed work.

Information Codex will discover or need

Locally: file structure, size, header variants, Python version, free disk, RAM, repository instructions. For cloud integration: workspace URL, runtime/compute, granted catalog/schema/volume, source table names, and allowed job schedules. Missing cloud access should not block local inventory and pilot preparation.

References

1. DuckDB Parquet reading/writing
2. DuckDB partitioned writes
3. Databricks CLI volume file operations
4. Files in Unity Catalog volumes
5. Databricks job bundles tutorial

This is an implementation plan, not a claim that the local archive or class workspace has been inspected. Example paths and cloud identifiers must be validated in the executing environment.